

Checkliste Data Leakage

1. Train/Test-Split

- Wurde der Split **ganz am Anfang** gemacht, bevor irgendwelche Transformationen (Skalierung, Normalisierung PCA, Feature-Selektion, Imputation) angewandt wurden? → LEAKAGE
- Wurde **Data Augmentation** vor dem Split durchgeführt? → LEAKAGE

2. Features prüfen

- Könnte ein Feature **direkt oder indirekt Informationen über das Label** enthalten? (z. B. „Diagnosecode“ bei Vorhersage von Krankheiten) → LEAKAGE
- Enthält ein Feature **zukünftige Informationen**, die im echten Vorhersagezeitpunkt noch nicht verfügbar wären? (z. B. „Anzahl verpasster Zahlungen“ bei Kreditausfallvorhersage) → LEAKAGE
- Sind Zeitspannen und Aggregationen korrekt berechnet, ohne Daten aus der Zukunft einzubeziehen? → KORREKT

3. Cross-Validation

- Ist die CV-Strategie korrekt für Zeitreihen (kein „in die Zukunft schauen“)? → KORREKT

Beispiel für korrektes Vorgehen:

```
# Normalisierung mit StandardScaler

scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)

X_test = scaler.transform(X_test)
```

Kernfrage:

„Hätte ich diese Information im echten Vorhersagezeitpunkt auch schon gehabt?“

Wenn die Antwort **nein** ist → LEAKAGE

Der Schlüssel zur Vermeidung liegt darin, eine strikte Trennung zwischen Trainings- und Testdaten in *allen* Phasen des Modellentwicklungsprozesses aufrechtzuerhalten. Das Testset sollte als "unberührbares", zukünftiges Datenset behandelt werden, das erst ganz am Ende zur finalen Evaluation herangezogen wird.

Skalierung & Normalisierung:

- ☐ Den Scaler (z.B. **MinMaxScaler**) **nur** auf den Trainingsdaten fitten
- ☐ Den gefitteten Scaler nutzen, um die Trainingsdaten zu transformieren
- ☐ Denselben Scaler nutzen, um die Testdaten zu transformieren
- **Niemals** den Scaler auf den Testdaten neu fitten.

Umgang mit fehlenden Werten (Imputation):

- ☐ Die Strategie zur Füllung fehlender Werte (z.B. Berechnung des Mittelwerts, Medians) **nur** aus den Trainingsdaten ableiten
- ☐ Fehlende Werte in den Trainings- **und** den Testdaten mit dem gelernten Wert füllen

Feature-Auswahl:

- ☐ Die Analyse, welche Features am wichtigsten sind (z.B. durch Korrelationsanalysen mit der Zielvariable, Feature Importance), wird **nur** auf den Trainingsdaten durchgeführt.

Encoding kategorialer Variablen:

- ☐ Den Encoder (z.B. **OneHotEncoder**) **nur** auf den Trainingsdaten fitten.
- ☐ Trainings- und Testdaten mit dem gefitteten Encoder transformieren.

Prinzip Zeitreihe: Man darf **nicht zufällig mischen**, sonst „sehen“ Trainingsdaten die Zukunft.

Für Zeitreihen muss eine spezielle Form der Kreuzvalidierung verwendet werden, die die chronologische Reihenfolge der Daten respektiert. Die gängigste Methode wird oft **Time Series Split**, **Forward-Chaining** oder **Rolling-Origin Cross-Validation** genannt.

Das Prinzip ist einfach: Das Modell bewegt sich auf dem Zeitstrahl immer nur vorwärts.

1. **Split 1:** Trainiere mit Daten aus Monat 1, teste auf Monat 2.
2. **Split 2:** Trainiere mit Daten aus Monat 1-2, teste auf Monat 3.
3. **Split 3:** Trainiere mit Daten aus Monat 1-3, teste auf Monat 4.
4. ... und so weiter.